

Artifact Description

Associated Paper

The Non-Linear Effects of Multi-Connector Retries in Microservice Architectures

Carlos Aderaldo and Nabor Mendonça

Open Science Badge Applied For

Reproducible Results. The core results reported in the paper can be reproduced from this artifact using the provided scripts and data. The artifact bundles the raw experimental output, an analysis notebook that processes it, and the resulting datasets and figures, so that every figure in the paper can be regenerated on a commodity machine.

Persistent Link

The artifact is archived on Zenodo and is permanently available at:

<https://zenodo.org/records/21364082>

Purpose

The paper presents an architecture-centric empirical study of how concurrent retry mechanisms across multiple connectors affect the resilience and performance of a Kubernetes-based microservice application. Using ResilienceBench-Operator to orchestrate experiments on a modified version of Google's Online Boutique demo, the study compares four benchmark configurations along the checkout request path under varying workloads, fault rates, and gRPC retry parameters.

This artifact serves two goals:

1. **Reproduce the paper's results (recommended path).** Re-run the analysis notebook over the already-collected raw data to regenerate the processed datasets and every figure presented in the paper.
2. **Re-run the full empirical study (optional path).** Re-execute the experiments end-to-end on a Kubernetes cluster to regenerate the raw data from scratch.

Contents

Component	Description
<code>experiment-config/</code>	Kustomize manifests for the experiments: the modified Online Boutique application, the benchmark CRD (<code>benchmark.yaml</code>), the workload definition, and the k6 load-testing script (<code>k6.js</code>). Enables re-running the experiments end-to-end.
<code>raw/</code>	Untouched JSON output produced by each experimental run of ResilienceBench-Operator. These files are the input to the analysis notebook.
<code>figures/data-processing-and-charts.ipynb</code>	Jupyter notebook that reads the raw runs, processes and normalizes them into the analysis datasets, and renders the paper's Pareto-front and parameter-profile (radar) charts as PDFs.
<code>dataset/</code>	Processed, analysis-ready CSV files produced by the notebook: <code>baseline.csv</code> , <code>retry.csv</code> , and <code>pareto.csv</code> .
<code>figures/</code>	The PDF charts that correspond to Figures 2–6 of the paper.
<code>README.md</code>	Full documentation, including environment setup, dependencies, a step-by-step reproduction guide, and a complete data dictionary.

Reproducing the Core Results

The recommended path is lightweight and runs on any machine:

1. Install Python 3.10+ and Jupyter, then install the dependencies:

```
python3 -m venv venv
source venv/bin/activate
pip install pandas numpy matplotlib jupyter
```

2. Open `figures/data-processing-and-charts.ipynb` and run all cells, top to bottom.

Running the notebook reproduces:

- `dataset/` — `baseline.csv` , `retry.csv` , and `pareto.csv` . Because the committed CSV files were generated by this exact pipeline, re-running the notebook overwrites them with byte-identical content.
- `figures/` — the Pareto-frontier scatter plots (**Figs. 2–4**), the combined Pareto overlay, and the radar charts of Pareto-optimal retry parameters (**Figs. 5–6**).

These figures directly answer the paper's two research questions:

- **RQ1 — Impact of multi-connector retries** (Figs. 2–4): frontend-only retries yield flat frontiers near the baseline; checkout-only retries shift the frontier toward higher success rates; and dual-connector retries expand the set of Pareto-optimal solutions while adding only modest gains over the best checkout-only configuration.
- **RQ2 — Effect of parameter variation** (Figs. 5–6): the radar charts reveal scattered profiles for frontend-only retries, compact and stable profiles for checkout-only retries, and a more diverse, harder-to-tune space for dual-connector retries.

Full instructions, the optional cluster-based re-execution path, and the complete data dictionary are provided in `README.md`.